PinLearn

Project Transformation Report

What changed, why it matters, how to explain it in an interview, and what to learn next.

Architecture | Security | Testing | CI/CD & Docker | UX & Accessibility | Interview Prep

React 19 · Express 5 · MongoDB · Zod · Pino · GitHub Actions

October 2026

CONTENTS

Table of Contents

01	At a Glance	Summary of the transformation
02	Project Overview	What it does, old vs new architecture
03	Complete Change Log	Every change, grouped by category
04	Before vs After	The most important improvements, side by side
05	Interview Cheat Sheet	Concepts, short answers, follow-ups
06	Learning Roadmap	What to study next and where
07	Recruiter View	How the project reads to a hiring team
08	Portfolio Value	Resume lines, screenshots, demo script
09	Final Assessment	Scores before and after

How to read the scores in this document. All scores are reviewer judgement, not measurements. They compare the codebase before and after these changes. Treat them as a guide to where the project is strong and where it is still thin.

SUMMARY

At a Glance

PinLearn went from a working feature demo with a single 1,199-line server file and no tests, to a layered, tested, containerised application with a CI pipeline and a consistent UX foundation.

1,199 → 40
lines in `server.js`

1 → 40
automated tests (26 backend, 14 frontend)

0 → 3
CI jobs (backend, frontend, Docker)

~25 → 1
error-handling code paths

4
critical security holes closed

0 → 2
health endpoints (liveness, readiness)

The five changes that matter most

Change	Why it matters in one sentence
Layered backend	Each file now has one job, so logic can be tested and changed without touching HTTP code.
Zod validation + central errors	Every input is checked at the door, and every failure comes back in the same shape with a request ID.
Server-derived streaks	The product's core metric can no longer be faked by an empty POST request.
Route guards + 401 handling	Signed-out users are redirected, and expired sessions recover cleanly instead of breaking pages.
Tests + CI + Docker	Every push is verified automatically, and anyone can run the full stack with one command.

What is still open

- No refresh tokens or token revocation; the JWT lives for 7 days in `sessionStorage`.
- Integration tests stop before the database. Signup, boards and streak flows are not tested end to end.
- Rate limiting and the YouTube cache are in-memory, so they do not scale past one server instance.
- `Dashboard.jsx` (470 lines) and `Kanban.jsx` (921 lines) are still large; there is no shared button or form component library.
- Docker images were written but not built locally; CI builds them on the first push.

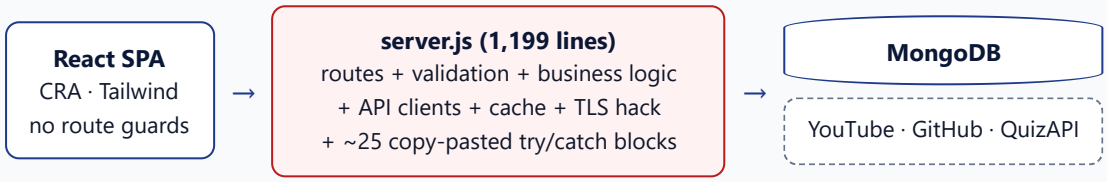
SECTION 1

Project Overview

What the application does

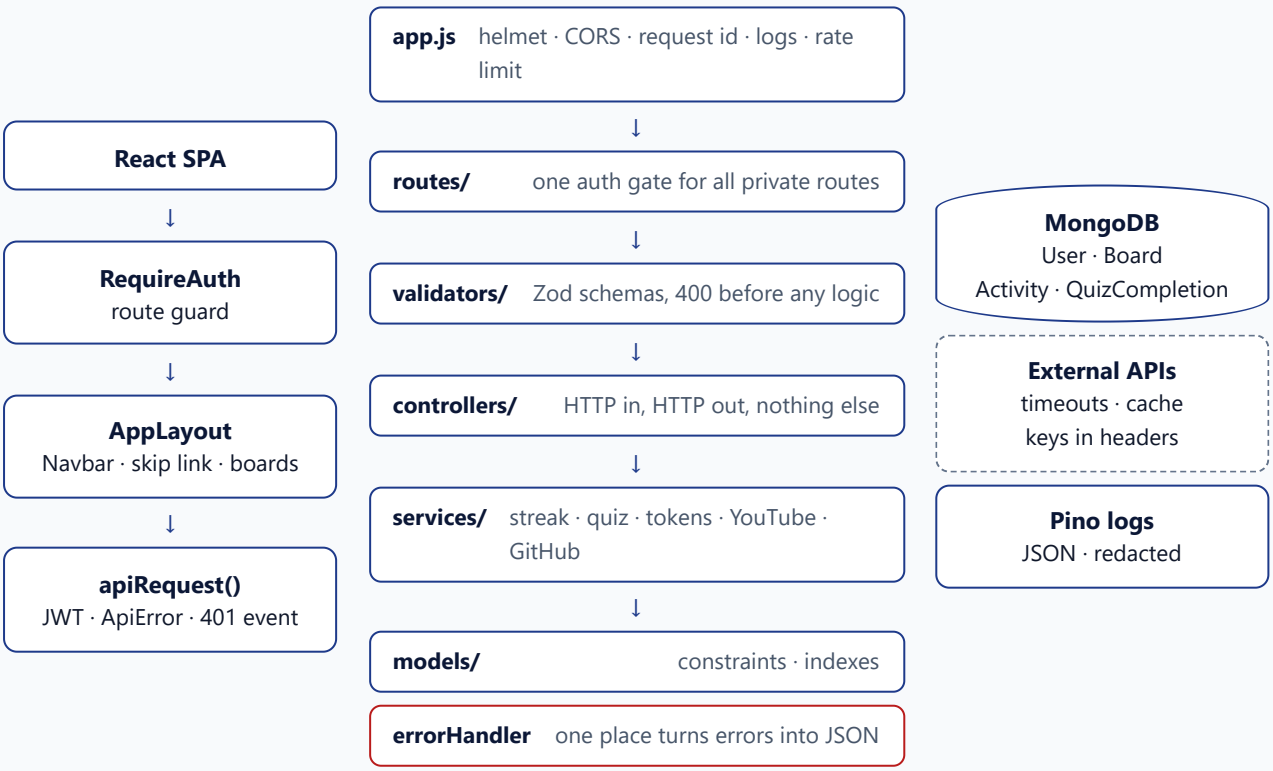
PinLearn is a Pinterest-style learning tracker. A user signs up and picks interests (React, DSA, DevOps...). The dashboard then surfaces YouTube tutorials and GitHub repositories for those topics. Users save resources to **boards**, move them through **saved** → **visited** → **in progress** → **completed**, attach **notes**, plan work on a **Kanban board**, take a **daily quiz**, and build a **streak** shown on an activity calendar.

Original architecture



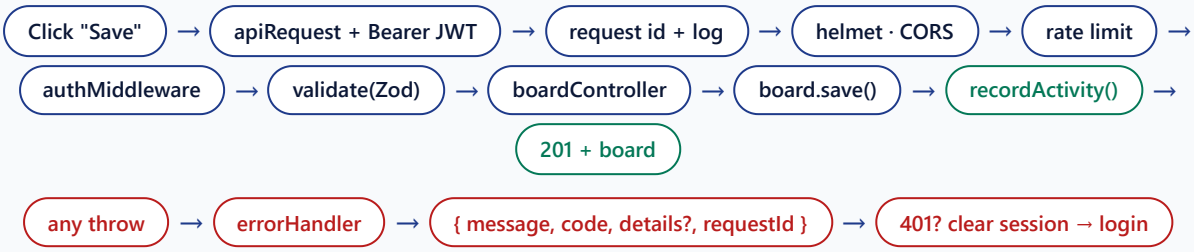
Before: everything in one file. Nothing could be tested in isolation, and render.yaml pointed at a health check that did not exist.

New architecture



After: requests flow down through layers with one responsibility each. Errors from any layer flow to a single handler.

Request flow (saving a resource)



Green: happy path. Red: any failure, from any layer, takes the same route back to the user.

Major technologies

Area	Technology	Role
Frontend	React 19, React Router 6, Redux Toolkit, Tailwind 3, @hello-pangea/dnd	UI, routing, theme state, Kanban drag and drop
Backend	Express 5, Mongoose 9, bcrypt, jsonwebtoken	API, data access, password hashing, auth tokens
Validation	**Zod 4** (new)	Schemas for every request and for environment config
Observability	**Pino, pino-http** (new)	Structured JSON logs with request IDs and redaction
Security	helmet, express-rate-limit, cors	Headers, throttling, origin policy
Testing	**node:test, supertest** (new), Jest, Testing Library	Backend unit + HTTP tests, frontend component tests
Delivery	**GitHub Actions, Docker, docker-compose, nginx** (new), Render, Vercel	CI, containers, hosting

Key engineering decisions

Validate at the boundary, trust inside

Zod parses input once in middleware. Controllers read `req.valid` and never re-check types.

Pure functions for business rules

`computeStreak()` and `scoreQuiz()` take data and return data, so they are tested without a database.

The server derives business events

Streak days are written only after verified actions. The client can no longer claim activity.

Let the database enforce uniqueness

A unique index closes the "check then insert" race that application code alone cannot.

Fail closed in production

Empty `CORS_ORIGINS` rejects browsers; the self-signed TLS bypass is ignored in production.

Document known limits

The README lists what is not solved yet. Honest trade-offs read better than hidden gaps.

SECTION 2

Complete Change Log

Every change, grouped by category. Each entry ends with a one-line talking point you can say out loud.

Architecture Changes

Split the monolith into layers

`backend/{app,server}.js, config/, routes/, controllers/, services/, middleware/, validators/, utils/`

Before	One 1,199-line `server.js` holding routing, validation, logic, API clients, caching and startup.
Now	Eight folders, each with one responsibility. `server.js` is ~40 lines of startup and shutdown.
Why	Code that does everything cannot be tested, reviewed or changed safely.
Benefits	Isolated unit tests, smaller diffs, obvious place for every new feature.
Recruiter impact	Removes the single biggest red flag in the original review.
Talking point	*"I refactored a monolith into routes, controllers and services so business rules became pure, testable functions."*

App factory separated from server start

`backend/app.js, backend/server.js`

Before	Importing the app connected to MongoDB and started listening.
Now	`createApp()` builds the app; `server.js` connects, listens and handles shutdown.
Why	Tests need the app without a port or a database.
Benefits	supertest drives the real middleware stack in memory.
Recruiter impact	Shows awareness of testability as a design goal.
Talking point	*"Separating construction from startup is what made HTTP-level tests possible."*

Backend Changes

Centralised error handling

`middleware/errorHandler.js, utils/AppError.js`

Before	~25 copy-pasted try/catch blocks; responses mixed `{message}` and `{error}`; CORS errors became HTML pages.
Now	Code throws `AppError(status, message, code)`. One handler maps Zod, Mongoose, bad JSON, oversize body and duplicate-key errors to `{ message, code, details?, requestId }`.
Why	Inconsistent errors make clients fragile and leak internals.
Benefits	Predictable API contract; 5xx details never reach users; every error is traceable.
Recruiter impact	Signals production thinking rather than tutorial code.
Talking point	*"Express 5 forwards async errors, so handlers just throw and one middleware decides what the client sees."*

Schema validation on every endpoint

`validators/*.js, middleware/validate.js, config/env.js`

Before	Ad hoc checks; some routes validated, some (`/api/interests`) did not; no length limits.
Now	Zod schemas for every body, query and param, plus the environment itself. Results land on `req.valid` .
Why	Untrusted input is the root of most API bugs and injection attacks.
Benefits	Type coercion blocks `{"$ne": null}` injection; field-level error messages; config fails fast at boot.
Recruiter impact	Matches what reviewers expect in any production Node API.
Talking point	*"Validation happens once at the boundary; past that point the code can trust its inputs."*

Token logic in one service

`services/tokenService.js, middleware/authMiddleware.js`

Before	JWT signing and user-response shaping duplicated in signup and login.
Now	`signToken` , `verifyToken` (algorithm pinned to HS256) and `buildAuthResponse` in one file.
Why	Duplicated security code drifts; pinning the algorithm blocks algorithm-confusion attacks.
Benefits	One place to add refresh tokens later.
Recruiter impact	Shows DRY applied where it matters most.
Talking point	*"Auth logic lives in one service, and verification only accepts the algorithm we sign with."*

Third-party integrations as services

`services/{youtube,github,quiz}Service.js, services/providerClient.js`

Before	Inline fetch code; YouTube cache never evicted; substring filter dropped "shortcuts" tutorials; API key in URL.
Now	Bounded TTL cache (500 entries) with request coalescing; whole-word filtering; keys in headers; 8-second timeouts; pure `scoreQuiz()` .
Why	Unbounded memory growth and leaked keys are production incidents waiting to happen.
Benefits	Stable memory, fewer quota hits, testable scoring.
Recruiter impact	Demonstrates awareness of cost, quotas and failure modes.
Talking point	*"Concurrent identical searches share one in-flight request, and the cache is capped so memory cannot grow forever."*

Frontend Changes

Route guards and shared layout

`src/App.jsx, routes/RequireAuth.jsx, routes/AppLayout.jsx, routes/pageRoutes.jsx`

Before	Any URL rendered without a token; `<Navbar/>` copy-pasted into 9 routes; boards fetched on the login page.
Now	Nested routes: `RequireAuth` → `AppLayout` (Navbar + Outlet) → pages. Boards load only after login.
Why	Duplicated layout drifts, and unguarded pages show broken shells.
Benefits	One place for auth, one place for layout; fewer wasted requests.
Recruiter impact	Fixes an obvious gap any reviewer clicks into first.
Talking point	*"Layout routes with Outlet let me guard and wrap every private page in one declaration."*

Global 401 handling

`src/api/client.js`

Before	Expired tokens produced generic errors on each page.
Now	A 401 on an authenticated call clears the session and fires `auth:session-expired`; the guard redirects to login. Errors are typed `ApiError` with status and code.
Why	Session expiry is a cross-cutting concern; handling it per page guarantees inconsistency.
Benefits	Clean recovery; failed logins do not trigger it (tested).
Recruiter impact	Shows thinking about the whole session lifecycle.
Talking point	*"The API client owns session expiry, so no page has to know about it."*

Error boundary, cleanup and extraction

`components/ErrorBoundary.jsx, hooks/useThemeClass.js, Kanban/kanbanUtils.js, package.json`

Before	A render crash blanked the app; unused `redux-saga` and `dotenv`; Kanban helpers inside a 1,034-line component.
Now	ErrorBoundary fallback with reload; unused dependencies removed; Kanban helpers moved to `kanbanUtils.js` (921 lines remain).
Why	Blank screens lose users; dead dependencies suggest copy-paste.
Benefits	Graceful failure, smaller install, clearer component.
Recruiter impact	Removes "tutorial residue" signals.
Talking point	*"Error boundaries are still class components in React; I use one at the root as the last line of defence."*

Database Changes

Schema-level constraints

`models/User.js, models/Boards.js, models/QuizCompletion.js`

Before	Password selectable by default; email not normalised; unbounded strings and arrays.
Now	`password: select:false`; email lowercased and trimmed; max lengths everywhere; caps of 30 interests, 500 resources per board, 100 notes per resource.
Why	Defence in depth: the database should reject bad data even if a route forgets.
Benefits	No accidental password leaks; documents cannot approach Mongo's 16 MB limit.
Recruiter impact	Shows understanding of embedded-document limits.
Talking point	*"Secure defaults: the password field is excluded unless a query explicitly asks for it."*

Indexes and the board-name race

`models/Boards.js, controllers/boardController.js`

Before	Duplicate check with a case-insensitive regex, then insert: unindexable and racy.
Now	Case-folded `nameKey` with a partial unique index `{userId, nameKey}`; duplicate-key error mapped to 409; index on `{userId, createdAt}`.
Why	Two parallel requests could both pass the check and both insert.
Benefits	Correct under concurrency; faster board listing.
Recruiter impact	Race conditions are a classic interview topic; this is a real example.
Talking point	*"The partial index meant legacy boards without the new field could not break the migration."*

Dedicated Activity model — `models/Activity.js, services/streakService.js`

Before	Non-quiz activity was stored as a fake `QuizCompletion` with `quizId: "activity-save"`.
Now	`Activity {userId, date, type}` with a unique `{userId, date}` index; streak reads both collections.
Why	Overloading a model hides meaning and corrupts analytics.
Benefits	Idempotent daily writes; honest data model; backward compatible.
Recruiter impact	Shows data-modelling judgement.
Talking point	*"A unique index makes 'record today's activity' idempotent without any locking."*

Security Changes

Change	Before	Now	Talking point
Paid API proxies protected routes/contentRoutes.js, middleware/rateLimiters.js	Anyone could call `/api/youtube` and `/api/github` and burn quota.	Auth required + 30 req/min per user; `trust proxy` in production.	*"Rate-limit by user, not IP, when everyone sits behind one proxy."*
Streak farming removed controllers/board, resource, quiz	Empty `POST /api/activity` granted a streak day.	Route deleted; activity recorded only after verified actions.	*"Never let the client assert a business event."*
URL sanitising utils/url.js, validators/boards.js	`javascript:` URLs could be stored and rendered as links.	Only `http(s)` accepted and normalised.	*"React escapes text, not href schemes."*
Credential hardening controllers/authController.js	bcrypt cost 10, no max length, timing revealed unknown emails.	Cost 12, 72-char cap, dummy-hash compare for unknown emails.	*"Constant-ish timing prevents account enumeration."*
Fail-closed config config/cors.js, config/env.js, utils/logger.js	Empty CORS allowed all origins; TLS bypass usable anywhere; tokens could reach logs.	CORS rejects in prod when empty; TLS bypass dev-only; auth headers and passwords redacted.	*"Secure defaults beat secure documentation."*
Debug route removed routes/*	`GET /api/protected` shipped to production.	Removed, with a test asserting 404.	*"Regression tests stop it coming back."*

UX Changes

Change	Before	Now	UX principle
Navigation state Navbar/Navbar.jsx, routes/AppLayout.jsx	No active link; logo not clickable; mobile menu stayed open.	Active underline + `aria-current`; logo links home; menu closes on navigate/Escape; skip link.	Visibility of system status; WCAG 2.4.1
Loading skeletons components/ui/Skeleton.jsx	"Loading..." text or a bare div.	Card-shaped placeholders with `role="status"` on Boards, Topic, Quiz, Profile.	Perceived performance; no layout shift

Empty and error states components/ui/StateViews.jsx	Three different styles; no retry.	Shared `EmptyState` / `ErrorState` that explain the next step.	Empty states teach; errors offer a way out
404 and titles ui/NotFound.jsx, hooks/usePageTitle.js, public/index.html	Blank page on bad URLs; every tab said "React App".	404 with a way back; per-route titles; real meta description.	Recognition over recall
Accessible dialog Boards/Boards.jsx	No dialog role, unlabelled close button, no Escape.	`role="dialog"`, `aria-modal`, labelled heading, Escape and click-outside close.	WCAG 4.1.2 Name, Role, Value
Responsive fixes index.css, Kanban, BoardDetails, Dashboard	Fixed min-widths caused horizontal scroll on phones.	Fluid widths; 2-column Kanban on tablets; media capped at 100%.	Mobile-first layout

Testing Changes

Backend unit and HTTP tests (26)

`backend/tests/{streak,units,api}.test.js`

Before	`"test": "echo Error: no test specified"`.
Now	Node's built-in runner. Unit: streak edge cases (month/year boundaries, timezones, gaps), quiz scoring, URL sanitiser, schemas. HTTP: auth, validation, error shape, removed routes, 404, 503 for unconfigured integrations.
Why	Untested code cannot be refactored safely.
Benefits	Fast, hermetic (local `.env` is skipped in tests), no database needed.
Recruiter impact	Turns the weakest score (1/10) into a credible story.
Talking point	*"Pure functions get unit tests; the HTTP layer gets supertest tests that reject before touching the DB."*

Frontend tests (14)

`src/App.test.js, api/client.test.js, components/ErrorBoundary.test.jsx`

Before	One default CRA test that rendered `App` without its Redux provider.
Now	Route-guard redirects for 5 private routes, 404 page, active nav + skip link + page title, 401 handling (4 cases), ErrorBoundary fallback.
Why	Tests should describe behaviour a user or a bug report would mention.
Benefits	Guards against regressions in auth flow and accessibility.
Recruiter impact	Shows Testing Library's "test like a user" approach.
Talking point	*"I query by role and accessible name, so my tests also check accessibility."*

CI/CD Changes

GitHub Actions pipeline

`.github/workflows/ci.yml`

Before	No automated checks.
Now	On every push and PR: backend `npm ci` + tests + `npm audit`; frontend tests + production build (lint warnings fail); then both Docker images build.
Why	Humans forget to run tests; pipelines do not.
Benefits	Broken code cannot reach `main` silently; dependency CVEs surface early.
Recruiter impact	A green CI badge is one of the fastest credibility signals on GitHub.
Talking point	*"Docker builds only run after both test jobs pass, so we never package broken code."*

DevOps Changes

Containers and local stack

`backend/Dockerfile, Pinterest/Dockerfile, Pinterest/nginx.conf, docker-compose.yml, backend/.env.example`

Before	Manual setup; secrets list only in prose.
Now	Multi-stage images; API runs as non-root with a HEALTHCHECK; web served by nginx with SPA fallback and long-lived static caching; compose runs API + web + MongoDB.
Why	"Works on my machine" is not a deployment strategy.
Benefits	One-command setup; smaller images; reproducible builds.
Recruiter impact	Docker literacy is expected for most backend roles.
Talking point	*"CRA inlines env vars at build time, so the API URL is a Docker build arg, not a runtime env."*

Production Readiness Changes

Change	Before	Now
Health and readiness app.js, render.yaml, DEPLOYMENT.md	render.yaml checked `/api/health`, which did not exist.	`/api/health` (liveness) and `/api/ready` (503 when MongoDB is down), both ahead of the rate limiter.
Structured logging utils/logger.js, app.js	`console.error` with full provider bodies.	Pino JSON logs, request IDs echoed as `X-Request-Id`, redaction of auth headers and passwords.
Lifecycle server.js, config/env.js	No shutdown handling; env checked for two keys.	Graceful SIGTERM/SIGINT shutdown, unhandled-rejection handler, full Zod env validation.
Documentation README.md, DEPLOYMENT.md	Create React App boilerplate README.	Architecture diagram, setup, API table, design decisions, security notes, known limitations.

SECTION 3

Before vs After

Six changes that best show the jump in quality. Snippets are trimmed to the essential lines.

1. Error handling

BEFORE · REPEATED ~25 TIMES

```
app.post("/api/resources/note", auth, async (req,
res) => {
  try {
    if (!boardId) return res.status(400)
      .json({ error: "Missing boardId" });
    ...
    if (!board) return res.status(404)
      .json({ message: "Board not found" });
  } catch (error) {
    console.error("Error adding note:", error);
    res.status(500).json({ message: "Failed" });
  }
});
```

AFTER

```
router.post("/resources/note",
  validate({ body: schemas.addNote }),
  resources.addNote);

async function addNote(req, res) {
  const { boardId, resourceId, content } =
req.valid.body;
  const { board, resource } =
    await findOwnedResource(req.userId, boardId,
resourceId);
  resource.notes.push({ content });
  await board.save();
  res.json({ message: "Note added", notes:
resource.notes });
}
```

maintainability · readability · consistent API · observability

2. Streak integrity

BEFORE

```
// Frontend, after any save or completion:
markActivity();   // POST /api/activity

// Backend: trusts the call, writes a fake quiz row
await QuizCompletion.create({
  quizId: "activity-save", score: 0, ...
});
```

AFTER

```
// Inside the save handler, after the DB write
succeeds:
await board.save();
await recordActivity(req.userId, "save");

// The public endpoint no longer exists (test
asserts 404).
// Streak math is a pure function:
computeStreak(activeDates, today);
```

security · data integrity · testability

3. Input validation

BEFORE

```
const selectedInterests =
  Array.isArray(req.body.selectedInterests)
    ? req.body.selectedInterests   // any types
    : [];                          // missing =
wipe
```

AFTER

```
const interests = z
  .array(z.string().trim().min(1).max(40))
  .max(30, "At most 30 interests are allowed");

const updateInterests = z.object({
selectedInterests: interests });
```

security · correctness · self-documenting

4. Board uniqueness under concurrency

BEFORE · CHECK-THEN-INSERT RACE

```
const existing = await Board.findOne({
  userId,
  name: new RegExp(`^${escaped}$`, "i")  // no
index
});
if (existing) return res.status(409)...
await Board.create({ userId, name });   // race
window
```

AFTER · DATABASE IS THE SOURCE OF TRUTH

```
boardSchema.index({ userId: 1, nameKey: 1 }, {
  unique: true,
  partialFilterExpression: { nameKey: { $type:
"string" } }
});

try { await Board.create({ userId, name }); }
catch (e) { if (e.code === 11000) throw
duplicateError(); throw e; }
```

correctness · scalability · performance

5. Frontend routing

BEFORE · REPEATED FOR 9 ROUTES, NO GUARD

```
<Route path="/boards" element={
  <>
    <Navbar />
    <Boards boards={boards} ... />
  </>
} />
```

AFTER

```
<Route element={<RequireAuth />}>
  <Route element={<AppLayout />}>
    <Route path="/boards" element={<BoardsRoute />}
/>
    ...
  </Route>
</Route>
<Route path="*" element={<NotFound />} />
```

security · maintainability · developer experience

6. Third-party cache

BEFORE · MEMORY LEAK

```
if (Date.now() > entry.expiresAt) {
  return null;          // expired entry stays
forever
}
...
`&key=${process.env.YOUTUBE_API_KEY}`  // key in
URL
```

AFTER

```
if (Date.now() > entry.expiresAt) {
  cache.delete(key); return null;
}
if (cache.size >= MAX_CACHE_ENTRIES)
  cache.delete(cache.keys().next().value);
headers: { "X-Goog-Api-Key": config.YOUTUBE_API_KEY
}
```

scalability · security · cost control

SECTION 4

Interview Cheat Sheet

Short answers you can give in 30 seconds, tied to this project. Practise saying them aloud.

What is centralised error handling?

Short answer	A single middleware receives every error, decides the status code and response body, and logs it. Route code just throws.
How we used it	`AppError(status, message, code)` plus `errorHandler.js`, which maps Zod, Mongoose, JSON-parse and duplicate-key errors to one JSON shape with a `requestId`.
Why industry uses it	Consistent contracts for clients, no leaked stack traces, one place to add alerting.
Follow-ups	How does Express know a middleware is an error handler? (four arguments) · What changed in Express 5 for async errors? · Operational vs programmer errors?

Why validate requests with a schema library?

Short answer	A schema declares the exact shape of valid input; anything else is rejected before business logic runs.
How we used it	Zod schemas for every body, query and param, plus `process.env`. A `validate()` middleware puts parsed data on `req.valid`.
Why industry uses it	Prevents injection, gives precise error messages, and doubles as documentation (and can generate OpenAPI or TypeScript types).
Follow-ups	Parse vs validate? · Where else should validation live (DB constraints)? · How does NoSQL injection work?

What is a service layer?

Short answer	A layer that holds business rules, separate from HTTP (controllers) and storage (models).
How we used it	`streakService`, `quizService`, `tokenService`, `youtubeService`. Controllers are 5-15 lines each.
Why industry uses it	Logic can be reused (CLI, queue worker, other routes) and unit-tested without HTTP.
Follow-ups	Service vs repository pattern? · Where would you put a transaction? · How would dependency injection help testing?

How do route guards work in React?

Short answer	A wrapper route checks auth and either renders its children (`<Outlet/>`) or redirects with `<Navigate/>`.
How we used it	`RequireAuth` wraps `AppLayout`, which wraps all private pages, and listens for a session-expired event.
Why industry uses it	One declaration protects many routes; layout and auth stay in sync.
Follow-ups	Is a client-side guard real security? (No: the API must enforce it) · How would you preserve the intended URL after login? · Role-based guards?

What is rate limiting and how do you choose the key?

Short answer	Capping requests per client per time window to prevent abuse and protect resources.
How we used it	Global 300/15 min, auth 20/15 min, provider proxies 30/min keyed by **user id**; `trust proxy` so IPs are real behind Render.
Why industry uses it	Stops brute force, scraping and runaway bills on paid APIs.
Follow-ups	Fixed window vs sliding window vs token bucket? · Why does in-memory limiting break with two servers? · What is a 429 + Retry-After?

What is structured logging?

Short answer	Logging JSON objects with fields instead of free text, so logs can be searched and aggregated.
How we used it	Pino with a per-request ID returned in `X-Request-Id`; secrets redacted; pretty output in dev only.
Why industry uses it	Tools like Datadog, ELK and CloudWatch query fields; request IDs connect a user's bug report to one log line.
Follow-ups	Logs vs metrics vs traces? · What must never be logged? · What is log level and when do you use each?

Liveness vs readiness checks?

Short answer	Liveness: "is the process alive?" Readiness: "can it serve traffic right now?"
How we used it	`/api/health` always 200 for Render; `/api/ready` returns 503 when MongoDB is disconnected. Both bypass the rate limiter.
Why industry uses it	Orchestrators restart dead containers and stop routing to unready ones, enabling zero-downtime deploys.
Follow-ups	Why not check the DB in liveness? (a DB outage would restart every pod) · What is graceful shutdown?

What is CI/CD?

Short answer	CI automatically builds and tests every change; CD automatically deploys changes that pass.
How we used it	GitHub Actions: backend tests + audit, frontend tests + build, then Docker builds. Render and Vercel deploy from `main`.
Why industry uses it	Small, verified, frequent changes are safer than big manual releases.
Follow-ups	How would you add a staging environment? · Branch protection? · How do you handle secrets in CI?

Why Docker, and what is a multi-stage build?

Short answer	Docker packages an app with its runtime so it runs the same everywhere. Multi-stage builds compile in one image and copy only the output into a small final image.
How we used it	API: install prod deps, run as non-root, HEALTHCHECK. Web: build with Node, serve static files with nginx.
Why industry uses it	Reproducible environments, smaller attack surface, standard unit for Kubernetes and cloud hosts.
Follow-ups	Image vs container? · Why order COPY package.json before COPY . ? (layer caching) · Why not run as root?

How do you test a Node API?

Short answer	Unit-test pure logic; integration-test HTTP behaviour through the real middleware stack; keep a few end-to-end tests.
How we used it	`node:test` for streak and scoring rules; supertest against `createApp()` for auth, validation and error shape.
Why industry uses it	The testing pyramid / trophy balances speed and confidence.
Follow-ups	How would you test DB flows? (mongodb-memory-server or test containers) · Mocks vs fakes? · What makes a test flaky?

How did you refactor a 1,199-line file without breaking it?

Short answer	Kept every URL and response shape, extracted by responsibility, and added tests at the HTTP boundary.
How we used it	Same routes, same JWT format, same login flow; frontend needed only two small changes (auth on content calls, removed `markActivity`).
Why industry uses it	Behaviour-preserving refactors behind tests are how teams pay down debt without big-bang rewrites.
Follow-ups	Strangler-fig pattern? · How do you refactor with no tests? (characterisation tests first) · What would you split next?

SECTION 5

Learning Roadmap

Not lessons: starting points. Each card tells you why a concept is worth your time and where to go next. Times are rough estimates for working proficiency.

1 · Validation → 2 · Error middleware → 3 · Unit + integration tests → 4 · GitHub Actions → 5 · Docker → 6 · JWT + refresh tokens → 7 · React Query → 8 · Redis caching + rate limits → 9 · Observability → 10 · Design systems + a11y

Suggested order: each step builds on the one before and maps to a change already in this codebase.

API Validation (Zod)

Beginner

What it is	Declaring the shape of data once and parsing untrusted input against it.
Why it is cool	One schema can validate requests, generate TypeScript types and produce API docs. Write once, use three times.
In this project	Every endpoint and the environment config; blocked NoSQL-injection payloads in tests.
Industry usage	tRPC, Next.js server actions, form libraries; standard in TypeScript backends.
Learn next	refine/transform · discriminated unions · z.infer types · zod-to-openapi
Resources	Docs: zod.dev · Blog: Total TypeScript (Matt Pocock) on Zod · Course: Total TypeScript Zod tutorial
Time	3-5 hours

Error Middleware

Beginner

What it is	A four-argument Express middleware that turns thrown errors into HTTP responses.
Why it is cool	It deletes code: dozens of try/catch blocks disappear and your API becomes predictable.
In this project	`AppError` + `errorHandler.js`; standard `{message, code, requestId}` body.
Industry usage	Every mature web framework has this hook (NestJS exception filters, Spring @ControllerAdvice).
Learn next	Operational vs programmer errors · RFC 9457 Problem Details · error tracking (Sentry)
Resources	Docs: expressjs.com/en/guide/error-handling.html · Blog: "Node.js Best Practices" repo (goldbergyoni) error section
Time	2-3 hours

Service Layer & Repository Pattern

Intermediate

What it is	Service: where business rules live. Repository: an object that hides how data is stored.
Why it is cool	Swap MongoDB for Postgres, or HTTP for a queue, and the business rules do not change.
In this project	Services exist; a repository layer does not yet (controllers call Mongoose directly). Good next refactor.
Industry usage	Clean / hexagonal architecture in large codebases; NestJS providers.
Learn next	Hexagonal architecture · dependency inversion · unit of work
Resources	Docs: martinfowler.com/eaaCatalog (Service Layer, Repository) · Book: "Patterns of Enterprise Application Architecture"
Time	1 week

Dependency Injection

Intermediate

What it is	Passing a component's dependencies in, rather than letting it import or create them.
Why it is cool	Tests can hand in a fake database or a fake clock. Our streak tests had to pass `today` in for exactly this reason.
In this project	Partial: `createApp()` and `computeStreak(dates, today)`. Services still import config directly.
Industry usage	NestJS, Angular, Spring and .NET are built around it.
Learn next	Constructor injection · IoC containers · composition root
Resources	Docs: docs.nestjs.com/providers · Blog: Martin Fowler, "Inversion of Control Containers and the Dependency Injection pattern"
Time	4-6 hours

JWT Authentication & Refresh Tokens

Intermediate

What it is	Signed tokens prove identity statelessly. Refresh tokens let access tokens be short-lived without forcing re-login.
Why it is cool	It is the difference between "a stolen token works for 7 days" and "a stolen token works for 15 minutes and can be revoked".
In this project	HS256-pinned 7-day JWT in sessionStorage. Refresh tokens are the top security item still open.
Industry usage	OAuth 2.0 / OpenID Connect providers (Auth0, Cognito, Google).
Learn next	httpOnly + SameSite cookies · token rotation · revocation lists · CSRF
Resources	Docs: jwt.io/introduction · OWASP JSON Web Token Cheat Sheet · YouTube: Hussein Nasser on JWT and sessions
Time	1-2 weeks to implement properly

Unit & Integration Testing

Beginner

What it is	Unit: one function in isolation. Integration: several pieces together, such as HTTP + middleware + validation.
Why it is cool	Tests are what let you refactor 1,199 lines without fear. They are your safety net and your documentation.
In this project	26 backend tests (node:test + supertest), 14 frontend tests (Jest + Testing Library).
Industry usage	Required for merge in almost every professional team.
Learn next	mongodb-memory-server · test doubles · coverage · Playwright end-to-end tests
Resources	Docs: nodejs.org/api/test.html, testing-library.com · Blog: Kent C. Dodds, "The Testing Trophy" · Course: TestingJavaScript.com
Time	1-2 weeks

GitHub Actions / CI/CD

Beginner

What it is	YAML workflows that run jobs (test, build, deploy) on repository events.
Why it is cool	Your repo gets a robot teammate that checks every change for free.
In this project	Three jobs with dependency caching and a gated Docker build.
Industry usage	GitHub Actions, GitLab CI, Jenkins, CircleCI.
Learn next	Matrix builds · environments + secrets · branch protection · deploy on tag
Resources	Docs: docs.github.com/actions · YouTube: TechWorld with Nana, GitHub Actions tutorial
Time	4-8 hours

Docker

Intermediate

What it is	Packaging an application and its runtime into an image that runs identically anywhere.
Why it is cool	`docker compose up` gives a new teammate the whole stack, including MongoDB, in one command.
In this project	Multi-stage API and web images, non-root user, HEALTHCHECK, nginx SPA config, compose file.
Industry usage	The deployment unit for Kubernetes, ECS, Cloud Run, Render and Fly.io.
Learn next	Layer caching · image scanning · volumes and networks · Kubernetes basics
Resources	Docs: docs.docker.com/get-started · YouTube: TechWorld with Nana, Docker tutorial · Course: Frontend Masters, "Complete Intro to Containers"
Time	1-2 weeks

Rate Limiting & Redis

Intermediate

What it is	Throttling clients per time window; Redis is the in-memory store that makes it work across many servers.
Why it is cool	The same Redis instance can back rate limits, caches, sessions and job queues.
In this project	Limits and cache are in-process today. Moving them to Redis is the main scalability step.
Industry usage	API gateways (Stripe, GitHub), CDNs, login protection.
Learn next	Token bucket · sliding window · rate-limit-redis · cache-aside and TTLs
Resources	Docs: redis.io/docs · Blog: Stripe engineering, "Scaling your API with rate limiters" · YouTube: Hussein Nasser on caching
Time	1 week

Logging, Monitoring & Observability

Advanced

What it is	Logs (what happened), metrics (how much, how fast) and traces (where time went across services).
Why it is cool	You can answer "why was this user's request slow?" in seconds instead of guessing.
In this project	Structured Pino logs with request IDs. No metrics, tracing or alerting yet.
Industry usage	Datadog, Grafana, New Relic, Sentry; OpenTelemetry as the open standard.
Learn next	OpenTelemetry · RED metrics · SLOs and error budgets · Sentry for frontend errors
Resources	Docs: getpino.io, opentelemetry.io/docs · Book: Google SRE book (free online) · Blog: Charity Majors on observability
Time	2-3 weeks

React Query (TanStack Query)

Intermediate

What it is	A library that manages server data: fetching, caching, refetching and mutation state.
Why it is cool	It would replace most of `useBoards` and the hand-written loading/error state across pages.
In this project	Not used yet. Server state is still hand-rolled in hooks and components.
Industry usage	Default choice for server state in modern React apps.
Learn next	Query keys · invalidation · optimistic updates · Suspense
Resources	Docs: tanstack.com/query · Blog: TkDodo's "Practical React Query" series
Time	1 week

Design Systems

Intermediate

What it is	Shared tokens (colours, spacing, type) and components (Button, Input, Dialog) used everywhere.
Why it is cool	Change the primary colour once and the whole app updates. Consistency becomes free.
In this project	Started: `Skeleton`, `EmptyState`, `ErrorState`. Buttons and inputs are still styled inline.
Industry usage	Atlassian Design System, Stripe, GitHub Primer, shadcn/ui.
Learn next	Design tokens in Tailwind config · variant APIs (cva) · Storybook
Resources	Docs: storybook.js.org, ui.shadcn.com · Site: designsystemsrepo.com
Time	1-2 weeks

Accessibility (a11y)

Beginner

What it is	Building interfaces that work for keyboard, screen-reader and low-vision users.
Why it is cool	Accessible markup is also better for SEO, testing (Testing Library queries by role) and every user.
In this project	Skip link, `aria-current`, dialog roles, status regions, focus rings, reduced motion.
Industry usage	Legal requirement in many markets; WCAG 2.2 AA is the usual target.
Learn next	Focus trapping · ARIA authoring practices · colour contrast · axe DevTools
Resources	Docs: w3.org/WAI/WCAG22/quickref, w3.org/WAI/ARIA/apg · Course: web.dev "Learn Accessibility"
Time	1 week

Database Indexing & Concurrency

Advanced

What it is	Indexes make lookups fast and can enforce uniqueness; concurrency control keeps parallel writes correct.
Why it is cool	One line of index definition fixed a race condition that no amount of application code could.
In this project	Partial unique index on board names; unique daily activity; compound indexes on userId.
Industry usage	Every high-traffic system; a staple of system-design interviews.
Learn next	explain() plans · compound index order (ESR rule) · transactions · optimistic locking
Resources	Docs: mongodb.com/docs/manual/indexes · Book: "Designing Data-Intensive Applications" (Kleppmann)
Time	2-3 weeks

SECTION 6

Recruiter View

Before these improvements

Concerns a reviewer raised

- 1,199-line `server.js` god file
- One broken default test; no CI; no Docker
- Deploy config pointed at a missing health endpoint
- Paid API endpoints open to anonymous callers
- Streak could be faked with one request
- No frontend route guards
- Leftover debug route and unused dependencies
- Boilerplate README

How it read

A strong **feature demo**: real features, deployed, unusually security-aware for a junior, but with no evidence the candidate could build software a team could maintain. Verdict: **SDE-1, borderline at mid-size companies**.

After these improvements

Concern	Status	Evidence
God file	**Resolved**	Layered backend; 40-line entry point
No tests / CI	**Resolved**	40 tests; three-job GitHub Actions pipeline
Broken deploy check	**Resolved**	`/api/health` + `/api/ready`
Open paid proxies, streak farming, URL injection	**Resolved**	Auth + per-user limits; server-derived activity; http(s)-only URLs
No route guards	**Resolved**	`RequireAuth` + tests for 5 routes
Token lifecycle	**Partial**	Algorithm pinned, 401 handled; no refresh or revocation
DB-backed tests	**Partial**	HTTP tests stop before the database
Large components	**Partial**	Kanban helpers extracted; Dashboard untouched
Horizontal scale	**Open**	In-memory limiter and cache

What level does it look like now?



The codebase now reads as a strong SDE-1 project, and gives you material to hold SDE-2-style design conversations.

Why not SDE-2 outright? SDE-2 signals usually include owning a system at scale: distributed state (Redis), a full auth lifecycle, DB-backed integration tests, metrics and alerting, and evidence of trade-offs made under real load. The codebase now *names* these gaps clearly, which is a mature signal, but has not yet *closed* them.

> **Important:** a recruiter will ask you to explain every change. The level this project earns depends on whether you can defend the decisions in Sections 3 and 4 without notes. Code that you cannot explain counts against you.

SECTION 7

Portfolio Value

Resume bullets

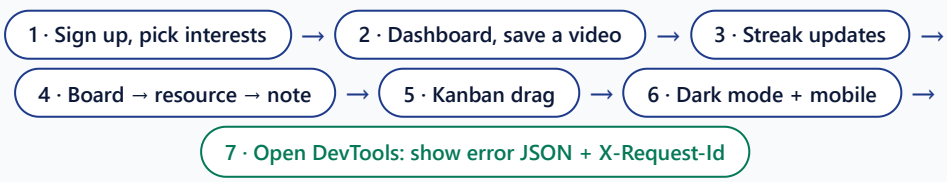
- Built **PinLearn**, a full-stack learning tracker (React 19, Express 5, MongoDB) with boards, notes, Kanban, quizzes and daily streaks, deployed on Vercel and Render.
- Refactored a 1,200-line Express monolith into a layered architecture (routes, controllers, services) with Zod validation and centralised error handling, enabling 26 backend tests.
- Closed four security issues, including unauthenticated paid-API proxies, client-forgeable streaks and stored `javascript:` URLs; added per-user rate limits and log redaction.
- Fixed a board-name race condition with a partial unique index; capped embedded arrays to stay within MongoDB document limits.
- Set up GitHub Actions CI (tests, build, audit, Docker) and multi-stage Docker images with liveness and readiness health checks.
- Added route guards, global session-expiry handling, an error boundary, skeleton loaders and WCAG-aligned navigation and dialogs.

Pick three or four bullets that match the role. Keep numbers you can defend in an interview.

Screenshots to include

Screenshot	What it proves
Dashboard (light + dark)	Visual polish, theming, streak calendar
Boards with progress	Core CRUD and progress tracking
Kanban mid-drag	Complex interaction (drag and drop)
Mobile nav open	Responsive design
Skeleton + empty state	Attention to UX states, not just happy paths
Green CI run	Engineering discipline
Architecture diagram	Systems thinking (reuse the one in the README)

Live demo script (5 minutes)



Step 7 is the differentiator: show a 400 validation error and the request ID, then explain the error pipeline.

Architecture decisions worth discussing

Why server-derived streaks?
Security thinking: trust boundaries and what the client can be allowed to claim.

Why a partial unique index?
Concurrency plus safe migration of existing data.

Why embed resources in boards?
Read pattern fits; caps protect the 16 MB limit; know when you would split them out.

Why sessionStorage and not cookies?
Honest trade-off: simpler, tab-scoped, but XSS-readable. Explain the refresh-cookie upgrade path.

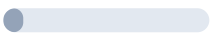
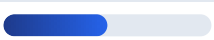
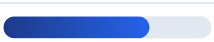
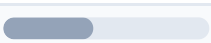
Why request coalescing?
Cost control on a quota-limited API; a small idea with a real-world payoff.

What breaks at 10× traffic?
In-memory limits and cache, embedded arrays, synchronous streak reads. Have the answer ready.

SECTION 8

Final Project Assessment

Reviewer judgement on a 10-point scale. "Before" is from the initial code review; "After" reflects the current codebase.

Area	Before		After	
Architecture	4		7.5	
Frontend	6		7	
Backend	5.5		8	
Testing	1		6	
Security	5.5		7.5	
Design / UX	6		7	
Scalability	3.5		5	
Production readiness	3.5		7	
Overall	**4.4**		**6.9**	

Why the scores are not higher

Testing (6)
Good unit and boundary coverage, but no DB-backed or end-to-end tests. Signup → save → streak is not tested as one flow.

Scalability (5)
Single-instance assumptions: memory-based rate limits and cache, no pagination on boards.

Frontend (7)
Large page components, no server-state library, CRA (deprecated) instead of Vite.

Security (7.5)
Long-lived JWT with no refresh or revocation; token readable by XSS.

Next five moves for the biggest gain

#	Move	Lifts
1	DB-backed integration tests with mongodb-memory-server	Testing, confidence
2	Short access token + httpOnly refresh cookie with rotation	Security
3	Redis for rate limits and the YouTube cache	Scalability
4	React Query + split Dashboard and Kanban into feature folders	Frontend, maintainability

5	Shared Button / Input / Dialog components with focus trapping	Design, accessibility

Bottom line. The project has moved from "a candidate who can build features" to "a candidate who builds software a team can maintain, test and deploy". Learn the concepts in Section 5 well enough to explain them without notes, and this becomes a project that carries interviews rather than just appearing on a resume.